

M15 / PROTECTED ANSWERS

M15 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	8
M15 EXPLAINED REVIEWS	8
DE15-01 Cloud fundamentals for data engineers - Worked recovery	8
What a strong answer proves	8
Worked reasoning	8
Failure diagnosis	8
Recovery decision	8
Changed retry - what I would look for	9
Teach it back	9
DE15-02 Identity, IAM and least privilege - Worked recovery	9
What a strong answer proves	9
Worked reasoning	9
Failure diagnosis	9
Recovery decision	9
Changed retry - what I would look for	10
Teach it back	10
DE15-03 Azure Storage concepts and OneLake - Worked recovery	10
What a strong answer proves	10
Worked reasoning	10
Failure diagnosis	10
Recovery decision	10

Changed retry - what I would look for 11

Teach it back 11

DE15-04 | Fabric workspace and item model - Worked recovery 11

What a strong answer proves 11

Worked reasoning 11

Failure diagnosis 11

Recovery decision 11

Changed retry - what I would look for 12

Teach it back 12

DE15-05 | OneLake paths, files, tables and shortcuts - Worked recovery 12

What a strong answer proves 12

Worked reasoning 12

Failure diagnosis 12

Recovery decision 12

Changed retry - what I would look for 13

Teach it back 13

DE15-06 | Fabric Lakehouse and Delta tables - Worked recovery 13

What a strong answer proves 13

Worked reasoning 13

Failure diagnosis 13

Recovery decision 13

Changed retry - what I would look for 14

Teach it back 14

DE15-07 Fabric notebooks - Worked recovery	14
What a strong answer proves	14
Worked reasoning	14
Failure diagnosis	14
Recovery decision	14
Changed retry - what I would look for	15
Teach it back	15
DE15-08 Spark in Fabric - Worked recovery	15
What a strong answer proves	15
Worked reasoning	15
Failure diagnosis	15
Recovery decision	15
Changed retry - what I would look for	16
Teach it back	16
DE15-09 Fabric Data Pipelines - Worked recovery	16
What a strong answer proves	16
Worked reasoning	16
Failure diagnosis	16
Recovery decision	16
Changed retry - what I would look for	17
Teach it back	17
DE15-10 SQL analytics endpoint and warehouse choice - Worked recovery	17
What a strong answer proves	17

Worked reasoning	17
Failure diagnosis	17
Recovery decision	17
Changed retry - what I would look for	18
Teach it back	18
DE15-11 Medallion architecture in Fabric - Worked recovery	18
What a strong answer proves	18
Worked reasoning	18
Failure diagnosis	18
Recovery decision	18
Changed retry - what I would look for	19
Teach it back	19
DE15-12 Incremental loads and MERGE in cloud - Worked recovery	19
What a strong answer proves	19
Worked reasoning	19
Failure diagnosis	19
Recovery decision	19
Changed retry - what I would look for	20
Teach it back	20
DE15-13 Permissions, connections and secrets - Worked recovery	20
What a strong answer proves	20
Worked reasoning	20
Failure diagnosis	20

Recovery decision	20
Changed retry - what I would look for	21
Teach it back	21
DE15-14 Monitoring, reliability and cost awareness - Worked recovery	21
What a strong answer proves	21
Worked reasoning	21
Failure diagnosis	21
Recovery decision	21
Changed retry - what I would look for	22
Teach it back	22
DE15-15 Git, deployment and environment separation - Worked recovery	22
What a strong answer proves	22
Worked reasoning	22
Failure diagnosis	22
Recovery decision	22
Changed retry - what I would look for	23
Teach it back	23
DE15-16 End-to-end Fabric data engineering case - Worked recovery	23
What a strong answer proves	23
Worked reasoning	23
Failure diagnosis	23
Recovery decision	24
Changed retry - what I would look for	24

Teach it back	24
-------------------------	----

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

M15 | EXPLAINED REVIEWS

Open only after you have saved a genuine attempt. These are lesson-specific recovery explanations, not a replacement for the independent task.

DE15-01 | Cloud fundamentals for data engineers - Worked recovery

What a strong answer proves

You can point at every box in a small cloud data-flow diagram and say what it stores, what executes there, who authorizes it, what it costs broadly, and what proof would show that it worked.

Worked reasoning

A regional courier company wants one platform for parcel scan files, transformations and analytics. The beginner answer is 'put everything in Fabric.' The engineering answer decomposes it: a workspace contains the lakehouse, notebooks and pipeline; OneLake holds durable data; Fabric compute executes Spark/SQL/pipeline work; Entra identities decide who may operate the workspace; capacity determines how much shared compute is available. The source system remains an external dependency. If the source API is down, adding more Fabric capacity does not repair the source. If a notebook lacks permission, restarting Spark does not grant authorization.

Write those components in a table with columns: component, owner, durable state, identity used, observable evidence, recovery action. This is a simple exercise, but it is the beginning of production reasoning.

Failure diagnosis

A notebook is slow and the capacity dashboard shows throttling. At the same time, a source API is returning 429 responses. Do not collapse these into one vague 'cloud is slow' incident. Separate source throttling from Fabric capacity pressure, record which stage waits, and test one hypothesis at a time. A common mistake is assuming SaaS means 'nothing to operate.' Fabric removes server patching and cluster provisioning work, but your team still owns bad joins, duplicate loads, over-broad access, runaway scans, broken dependencies and untested deployments. Another mistake is thinking a green pipeline run proves business correctness.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Change the source from an Azure database to an on-premises database behind a firewall. Add only the new connectivity/gateway/private-access concern; keep the rest of the architecture stable. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-02 | Identity, IAM and least privilege - Worked recovery

What a strong answer proves

You can explain why 'Contributor everywhere' is dangerous, distinguish workspace roles from Azure resource roles, and reason about effective permissions without guessing.

Worked reasoning

Suppose Priya develops a notebook that reads a bronze lakehouse and writes silver tables. She needs edit/run access in the development workspace, but an analyst who consumes gold output may only need Viewer/read permissions. A deployment service principal may need permission to update supported Fabric items without needing interactive human access.

In the supplied simulator, create three assignments: Analyst -> Viewer at workspace DEV; Engineer -> Contributor at workspace DEV; DeploymentSP -> Contributor at workspace TEST. Then ask whether Analyst can delete a notebook, whether Engineer can edit it, and whether DeploymentSP can modify TEST. The result should follow from principal + role + scope, not from the person's job title.

Failure diagnosis

A pipeline worked yesterday under your personal account but fails when scheduled under a service identity. That is valuable evidence: the code path is probably valid, while the runtime identity lacks some required permission or connection. Compare effective identities and scopes before editing transformation logic. Two beginner failures recur. First, giving a human account broad Owner/Admin access because a lab is blocked. Second, confusing Microsoft Entra directory roles, Azure RBAC roles and Fabric workspace roles. They interact with different resource planes. Fix the missing permission at the layer that actually denies the action.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

A contractor joins for two weeks to troubleshoot a read-only gold-table issue. Redesign your assignment so the contractor cannot edit notebooks, pipelines or security configuration. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-03 | Azure Storage concepts and OneLake - Worked recovery

What a strong answer proves

You can decide whether a dataset belongs in Files or Tables, read a OneLake path without panic, and explain how OneLake differs from but interoperates with Azure storage concepts.

Worked reasoning

Create a path inventory for a lakehouse named ops_lh in workspace de_dev. Raw CSV lands under Files/bronze/orders/2026/09/14/. A curated Delta table is exposed under Tables/silver_orders. Write the human-readable logical path and the ABFS-style shape. Do not invent a local C: drive path; OneLake is remote service storage.

The practice script validates that a path has a workspace, item name/type and Files/Tables boundary. It does not pretend to authenticate to OneLake. That separation is intentional: path reasoning can be tested locally; actual Entra-authenticated access remains learner-PC/Fabric proof.

Failure diagnosis

A notebook can read a file path but the SQL analytics endpoint cannot see it as a table. That is not automatically a permission bug. Check whether the data is actually represented as a supported Delta table in the lakehouse Tables area or exposed through an appropriate shortcut. Do not assume anything placed under Files automatically appears as a SQL-queryable table. Conversely, do not treat the Tables area as a dumping ground for random CSVs. Another mistake is hard-coding workspace/item display names into production code without a configuration strategy for DEV/TEST/PROD.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Move the raw CRM files to an external ADLS Gen2 account and use Fabric only for silver/gold. Explain what path/reference and identity assumptions change. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-04 | Fabric workspace and item model - Worked recovery

What a strong answer proves

You can inspect a Fabric workspace and explain what each item does, what it depends on and why workspace placement matters for security and deployment.

Worked reasoning

Build a DEV workspace inventory with six items: bronze lakehouse, curated lakehouse, ingest pipeline, transform notebook, Fabric environment and validation notebook. For each item record owner, upstream dependencies, downstream consumers and whether its definition should be version controlled.

Then compare two architectures: one giant workspace containing DEV/TEST/PROD suffixes versus separate workspaces per environment. The second usually gives clearer promotion and access boundaries. The lesson is not that one pattern is universally correct; it is that environment boundaries should be deliberate.

Failure diagnosis

A notebook deploys successfully to TEST but opens with a broken default lakehouse reference. The notebook file exists, so deployment 'succeeded,' yet the dependency binding is wrong. Inspect logical/physical item bindings instead of rerunning the same deployment. A common anti-pattern is storing experimental, production and personal artifacts in one workspace because it is convenient. Another is assuming a workspace role automatically grants every data-plane permission in every connected external system.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Split the design into DEV and PROD workspaces. Identify which dependencies must be rebound or configured differently after promotion. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-05 | OneLake paths, files, tables and shortcuts - Worked recovery

What a strong answer proves

You can explain when a shortcut is elegant and when a physical copy is justified, including the security and availability trade-off.

Worked reasoning

A sales team already has curated Delta data in a central lakehouse. Finance wants to query the same data. Option A copies 200 GB nightly into a finance lakehouse. Option B creates a OneLake shortcut to the curated table/folder. If Finance only needs read access and the source ownership/SLA are acceptable, the shortcut may avoid duplication and preserve one source of truth. If Finance needs an independently frozen month-end snapshot, a shortcut alone may not satisfy the requirement.

Run the shortcut decision script with fields `copy_required`, `independent_retention`, `source_sla_acceptable` and `source_format_delta`. The output is a recommendation plus reasons, not a magical product rule.

Failure diagnosis

A table exposed through a shortcut suddenly fails for one team while the source owner can still read it. Compare the consuming identity, shortcut configuration, source permissions and source health. Copying the data immediately may hide the real authorization problem and create uncontrolled duplicates. Do not promise independent disaster recovery from a shortcut to the same upstream data. Do not assume a shortcut changes the underlying file format into Delta. And do not forget that consumers still need permissions appropriate to the path through which they access the data.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Change the customer dimension requirement so PROD must continue operating for four hours even if the source workspace is unavailable. Revisit your decision. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-06 | Fabric Lakehouse and Delta tables - Worked recovery

What a strong answer proves

You can say exactly which engine writes the table, which surface queries it, and where the durable data lives. You do not call every Fabric SQL surface a warehouse.

Worked reasoning

Use the earlier orders domain. Raw CSV lands under Files/bronze/orders. A Fabric notebook reads it, validates schema, and writes a Delta table Tables/silver_orders. A gold table aggregates daily sales. Spark owns the transformation/write path. The SQL analytics endpoint exposes the Delta tables for read-oriented T-SQL analytics.

The supplied SQL file contains expected read queries and boundary comments. It intentionally does not pretend you can perform Spark-style Delta writes through the read-only lakehouse SQL endpoint. That distinction matters in interviews and real designs.

Failure diagnosis

Spark shows 1,000 new rows but a downstream SQL consumer does not immediately see what you expect. Verify that Spark wrote a valid lakehouse Delta table in the Tables area, inspect endpoint metadata/synchronization state, and compare counts by a stable boundary. Do not rewrite data merely to force visibility unless the root cause is understood. A frequent mistake is trying to INSERT/UPDATE lakehouse Delta tables through the SQL analytics endpoint as if it were a normal writable SQL database. Another is assuming an arbitrary external Delta location will automatically appear in the endpoint without supported registration/shortcut behavior.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

A consumer needs writable T-SQL transformations and stored procedures on the curated layer. Explain when a Fabric Warehouse becomes a better serving/transform choice than relying only on the lakehouse endpoint. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-07 | Fabric notebooks - Worked recovery

What a strong answer proves

You can open a notebook and quickly locate parameters, input contract, transformation logic, output write, validation and failure/recovery evidence.

Worked reasoning

The sample notebook pattern has five phases: parameters -> load -> validate -> transform/write -> reconcile/exit. A run_date parameter and source path are read once. The transformation function accepts data rather than reaching into global state. Before writing, the notebook checks required columns and rejects duplicates on the business key. After writing, it records row counts and a reconciliation status.

Open 07_notebook_patterns.py as ordinary Python to inspect the structure. In Fabric, translate the same shape into notebook cells. The goal is not to make local Python pretend to be Fabric; it is to practice the code organization before the cloud UI adds another layer.

Failure diagnosis

A scheduled notebook fails but an engineer says 'it works when I open it and run the last three cells.' That is evidence of hidden state or missing initialization. Restart the session and execute the documented path from the top with explicit parameters; a production notebook must not depend on yesterday's interactive state. Classic notebook failures include running cells out of order, leaving stale variables in memory, hard-coding PROD paths, installing libraries ad hoc in one session, printing secrets, and mixing exploratory display code with irreversible writes.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Change the notebook so a pipeline can pass `run_date` and `target_layer`. Add a safe failure response when `target_layer` is not bronze/silver/gold. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-08 | Spark in Fabric - Worked recovery

What a strong answer proves

You can identify the active Fabric runtime and explain why environment/runtime pinning matters for reproducibility and deployment.

Worked reasoning

As of this course build, Fabric Runtime 2.0 is GA with Apache Spark 4.1, Python 3.13 and Delta Lake 4.2. Microsoft states that it is not yet the default runtime and plans the default rollout for late September 2026. Therefore, the first cell of a serious lab records runtime/version evidence rather than assuming it.

The supplied runtime probe prints placeholders locally and documents the Fabric-side commands you should record: Spark version, Python version, attached environment and one tiny DataFrame action. Save that evidence with the lesson. Then compare it with M13's standalone/local baseline and explain why the numbers need not match.

Failure diagnosis

DEV works after an engineer manually installs a library in-session; PROD fails after deployment. The code commit is identical. Inspect environment/library configuration: the dependency lived only in interactive session state, not in a deployable environment definition. Do not pip-install random incompatible versions in a notebook just to force parity with a laptop. Do not assume the newest runtime is automatically default in every workspace. Do not diagnose every slow Spark job as a capacity problem without checking plan, data shape and shuffle behavior from M12.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Assume PROD remains on an older Fabric runtime while DEV opts into Runtime 2.0. List three tests you would require before promoting a notebook that uses a new Spark/Python feature. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-09 | Fabric Data Pipelines - Worked recovery

What a strong answer proves

You can read a Fabric pipeline and identify parameter flow, durable state, failure branches and the exact step that commits incremental progress.

Worked reasoning

Build the logical pipeline ``lookup_old_watermark -> lookup_new_watermark -> copy_increment -> validate -> commit_watermark``. The critical rule is the same pattern repaired in M08: do not commit progress before the copied/validated result is durable. If validation fails, the old watermark must remain so replay can re-read the failed interval.

Run `09_pipeline_incremental_simulator.py` locally. First it simulates a successful copy and commits the candidate watermark. Then it simulates a validation failure and proves that the committed watermark does not advance. This is a cloud-pipeline lesson grounded in a local deterministic control.

Failure diagnosis

The Copy activity succeeded, the notebook failed halfway through, and the next scheduled run finds no new rows. Investigate whether progress was committed too early. Restore the last safe watermark from evidence rather than manually increasing the source filter until rows 'look right.' Common failures: updating watermark immediately after source read, using one giant activity with hidden side effects, swallowing notebook failure and still running the commit step, or retrying a non-idempotent external side effect without a deduplication key.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Add a transient source timeout after the new watermark is read but before copy. Explain what may be retried without changing committed state. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-10 | SQL analytics endpoint and warehouse choice - Worked recovery

What a strong answer proves

You can choose lakehouse endpoint versus Warehouse based on write model, engine preference and serving requirements, not merely because both show a SQL editor.

Worked reasoning

Your silver and gold Delta tables are produced by Spark in a lakehouse. Analysts want T-SQL SELECT, views and a familiar connection endpoint. The SQL analytics endpoint fits. A separate finance workload requires SQL-first DDL/DML, stored procedures and curated dimensional transformations that the team wants to own in T-SQL. That requirement may justify a Fabric Warehouse for the gold serving layer.

Use 10_sql_endpoint_examples.sql to label each statement READ, ENDPOINT-OBJECT, or WRITE-TO-TABLE. The exercise is conceptual: statements that mutate lakehouse Delta table data through the endpoint are marked as the wrong surface.

Failure diagnosis

A developer tries to run UPDATE on a lakehouse Delta table through the SQL endpoint and gets blocked. The correct recovery is to use an appropriate Spark/Delta write path or reconsider whether the dataset should be owned by a Warehouse - not to disable security or export/re-import the table manually. Do not teach 'SQL endpoint = warehouse.' Do not promise full T-SQL DML against lakehouse Delta tables through the endpoint. Also do not create copies into Warehouse by default when the lakehouse endpoint already satisfies read analytics requirements.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

The BI team now needs a writable SQL staging layer before the final star schema. Redesign without creating two competing authoritative gold tables. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-11 | Medallion architecture in Fabric - Worked recovery

What a strong answer proves

You can define one-sentence grain, quality and allowed-consumer rules for each layer and explain how to replay Silver/Gold from preserved evidence.

Worked reasoning

For parcel scans, Bronze keeps source JSON/CSV or raw Delta with ingestion metadata. Silver enforces `parcel_id/event_time/event_type`, normalizes timestamps to UTC, deduplicates a defined event key and quarantines malformed records. Gold creates `current_parcel_status` and `daily_hub_volume` with documented grains.

The practice script reads a small contract registry and validates that Gold does not directly depend on the raw source. This is architectural lint, not a substitute for data tests, but it makes layer violations visible.

Failure diagnosis

A new source field breaks Silver parsing. Bronze still contains the original payloads. Pause promotion to Gold, update the Silver contract deliberately, replay the affected Bronze interval, reconcile counts and then resume. If Bronze had been overwritten, you would be dependent on the source resending history. Anti-patterns include transforming Bronze until it is no longer raw, letting reports query unstable Bronze directly, copying the same cleansed table into multiple Gold folders, and treating Gold as 'aggregated only.' A Gold table can be detailed if it is consumer-ready and contractually stable.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

A regulatory consumer requires immutable monthly snapshots. Add that requirement without turning every Gold table into a frozen copy. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-12 | Incremental loads and MERGE in cloud - Worked recovery

What a strong answer proves

You can draw candidate versus committed progress, name the idempotency key, and explain what happens on failure before and after each durable step.

Worked reasoning

A source orders table has LastModifyTime. The pipeline reads committed watermark 2026-09-13T00:00Z and current source maximum 2026-09-14T00:00Z. It copies rows (old, candidate] to a durable staging area, validates them, then a notebook MERGEs by order_id into the silver Delta table. Only after target reconciliation passes does the pipeline commit the candidate watermark.

The simulator also introduces a tied timestamp and replay. With a stable key/upsert, replaying boundary rows is acceptable; using > carelessly can skip a record that appears later with the prior max timestamp.

Failure diagnosis

A job failed after staging but before MERGE. The candidate watermark exists in logs but committed progress is unchanged. Safe recovery is to replay the interval from the old committed watermark, reconcile staging and apply idempotent MERGE. Do not manually commit the candidate because 'most rows probably made it.' Never set the watermark to source max before copied rows are durably saved and validated. Avoid assuming timestamps are unique/monotonic. A lookback window reduces some late-data risk but does not solve arbitrarily late corrections unless the source exposes a reliable change sequence/CDC contract.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Now assume `updated_at` can be corrected backwards by the source. Explain why the original watermark alone is insufficient and propose a safer source/change contract. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-13 | Permissions, connections and secrets - Worked recovery

What a strong answer proves

You can point to where identity is established, where secret material is stored, how it is retrieved, and what permissions allow that retrieval.

Worked reasoning

The bad pattern is `PASSWORD= 'P@ssw0rd! '` inside a notebook. The safer pattern is a managed Fabric connection or a Key Vault secret retrieved at runtime by an authorized identity. The notebook references a secret name, not the secret value. Logs record `connection=crm_prod`, `identity=svc-fabric-prod`, `result=success` without printing tokens.

The practice linter scans example configuration strings for suspicious hard-coded secret patterns and demonstrates a redacted configuration. It cannot prove cloud security, but it trains the habit that source files should remain safe to commit.

Failure diagnosis

A notebook fails with 403 when reading Key Vault after promotion. DEV worked under your personal identity. Compare PROD execution identity and Key Vault data-plane role assignment. Do not paste the secret directly into the notebook as a workaround. Never print secrets to confirm they are correct. Avoid storing them in notebook output, `.env` committed to Git, pipeline parameter defaults, screenshots or README files. Also understand that having permission to manage a Key Vault resource is not necessarily the same as permission to read secret values.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Replace a client-secret-based service principal with a managed/federated identity where the target supports it. Explain what secret disappears and what authorization still remains necessary. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-14 | Monitoring, reliability and cost awareness - Worked recovery

What a strong answer proves

You can distinguish job-level failure, Spark performance regression and shared-capacity pressure, and you know which Fabric monitoring surface to consult for each.

Worked reasoning

Suppose the nightly silver build normally finishes in 18 minutes and consumes a stable capacity footprint. It now takes 55 minutes. Start with run evidence: did source volume triple, did one activity retry, or did Spark time expand? Then inspect Spark jobs for skew/shuffle from M12. Finally inspect capacity metrics for overload/throttling and competing workloads.

The local simulator accepts `rows_processed`, `duration_seconds` and `retry_count` and calculates throughput plus a simple alert state. It intentionally does not claim to calculate Fabric billing; actual cost/consumption comes from Fabric metrics/pricing evidence.

Failure diagnosis

Gold is two hours stale but every pipeline run is green. Investigate scheduling gaps, source watermark movement and whether the pipeline processed zero rows due to a logic bug. Reliability requires semantic freshness checks, not just orchestration status. Do not respond to every slowdown by scaling capacity. A Cartesian join or tiny-file explosion can waste any capacity. Conversely, do not micro-optimize code while the capacity is visibly throttled by unrelated workloads. Also do not use a single successful run as a reliability claim.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

The job duration doubles but source volume is unchanged and capacity is healthy. List the next three pieces of Spark/data-shape evidence you would inspect before touching capacity size. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-15 | Git, deployment and environment separation - Worked recovery

What a strong answer proves

You can explain what Git tracks, what deployment moves, what remains environment-specific and how you would verify that PROD dependencies point to PROD resources.

Worked reasoning

Connect a DEV workspace to Git. Version the lakehouse metadata, notebook source and environment configuration that Fabric supports. TEST/PROD are separate workspaces. When promoting the notebook, ensure its default lakehouse/environment bind to the target equivalents rather than pointing back to DEV. Business table data is not 'deployed from Git' just because lakehouse metadata is versioned.

The simulator holds a mapping of logical names to DEV and PROD physical IDs. A validation function fails if the deployed notebook in PROD still references a DEV physical ID. This models one of the most important cloud-release checks without needing to fake a Fabric deployment locally.

Failure diagnosis

A PROD deployment succeeds, but the notebook reads DEV data. Stop the run, preserve evidence, inspect default-lakehouse binding and deployment rules, and correct the binding. This is a release-validation failure, not a reason to copy DEV data into PROD. Do not commit notebook outputs/secrets. Do not assume every Fabric item supports identical Git/deployment behavior. Do not treat a successful metadata deployment as proof that connections and downstream data contracts are correct. And never let PROD notebooks silently reference DEV lakehouses because an ID was hard-coded.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

The target workspace uses a different lakehouse name and connection. Design a logical mapping/deployment-rule approach instead of editing notebook source after every deployment. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.

DE15-16 | End-to-end Fabric data engineering case - Worked recovery

What a strong answer proves

You can narrate the architecture from one source change to one consumer query, including identity, durable state, validation, failure recovery, monitoring and deployment.

Worked reasoning

Case: a regional ecommerce company has an operational orders database and daily product files. DEV Fabric workspace contains bronze_lh, silver_gold_lh, ingest_orders pipeline, transform_orders notebook, validation notebook and a Fabric Environment. The pipeline reads the old/new watermark, copies changes to Bronze with run_id metadata, validates landing, calls the transform notebook to MERGE Silver, rebuilds affected Gold daily_sales partitions, reconciles counts/amounts, then commits progress. Analysts query Gold through the SQL analytics endpoint. A shortcut exposes a centrally owned product dimension without copying it. Secrets remain in managed connections/Key Vault.

Monitoring records last_success, source watermark, committed watermark, row counts, quarantine count, duration and capacity evidence. Git tracks supported definitions; deployment promotes to TEST/PROD with target bindings. A runbook states what to do if copy succeeds but transform fails: leave committed watermark unchanged and replay from the old boundary.

Run 16_end_to_end_local_case.py. It uses local CSV/JSON state to simulate two runs, including a replay. The expected proof is no duplicate order keys, committed watermark advances only after validation, and Gold total reconciles with Silver valid orders. This does not pretend to be Fabric execution; it proves the control logic before you reproduce it in Fabric.

Failure diagnosis

Production Gold is stale. The ingest pipeline says success, Silver is current, and Gold is not. The architecture narrows the incident to the Silver->Gold step or its scheduling/validation path. Because Bronze/Silver evidence and committed watermark are intact, you can repair Gold without re-pulling the source or guessing at missing data. A flashy portal screenshot is not portfolio proof if you cannot explain why each item exists. Avoid adding Eventstream, Warehouse, Dataflow, Databricks or AI components merely to increase tool count. Complexity must earn its place through a requirement.

Recovery decision

A strong recovery changes the smallest responsible boundary. Preserve the first failure, verify identity/environment and durable state, repair the root cause, rerun from the last safe boundary, and reconcile the corrected output. Do not use manual data edits or elevated permissions merely to make the symptom disappear.

Changed retry - what I would look for

Change the requirement: product data must remain in an external ADLS Gen2 account and finance requires a writable SQL gold mart. Update the design using a OneLake shortcut and Fabric Warehouse only where justified. The answer should explicitly name what changed, what remains invariant, and what evidence would prove the new design is correct.

Teach it back

Explain the lesson in under 90 seconds without service-name dumping. State the business requirement, the Fabric/Azure boundary used, the durable state, one likely failure and the evidence that proves recovery.